

Conceptos intermedios

13. Almacenamiento temporal

- **Stash** : "Estantería" o área de almacenamiento temporal donde Git guarda tus cambios actuales sin necesidad de crear un commit.
Se usa cuando necesitas limpiar tu área de trabajo rápidamente para cambiar de tarea, pero no quieres perder el progreso de lo que estabas programando en ese momento.
- **git stash** : Toma todos los cambios que tienes pendientes (tanto en el área de preparación como en el directorio de trabajo) y los guarda de forma segura en la estantería, dejando el proyecto totalmente limpio.
Se usa cuando surge una emergencia o un error prioritario en otra rama y necesitas cambiarte a ella de inmediato sin guardar lo que estás haciendo todavía.
- **git stash pop** : Recupera los últimos cambios que guardaste en la estantería y los vuelve a aplicar a tus archivos actuales, eliminándolos de la lista de pendientes.
Se usa cuando ya terminaste la tarea que te interrumpió y quieres retomar exactamente donde te quedaste con tu código anterior.
- **git stash list** : Muestra una lista numerada de todos los estados de código que has guardado temporalmente y que aún no has recuperado.
Se usa cuando has hecho varios "stashes" seguidos y necesitas identificar cuál es el que quieres recuperar para no aplicar los cambios equivocados.

14. Personalización

- **git config --global alias.st status** : Crea un alias llamado `st` para ejecutar el comando `git status`, permitiendo utilizar una versión abreviada del comando.
- **git config --global alias.lg "log --oneline --graph"** : Crea un alias llamado `lg` para mostrar el historial de commits de forma resumida y representado gráficamente mediante un árbol de ramas.
- **git config --global alias.co checkout** : Crea un alias llamado `co` para ejecutar el comando `git checkout`, facilitando el uso de una forma abreviada del comando.

15. Copia selectiva de cambios

- **git cherry-pick** : Instrucción general que le indica a Git realizar una integración selectiva de cambios, extrayendo el contenido de un commit específico de cualquier rama para aplicarlo como un nuevo commit en tu rama actual (HEAD).
Se utiliza principalmente para portar soluciones críticas (como hotfixes) entre diferentes líneas de tiempo sin necesidad de fusionar ramas completas, para recuperar trabajo realizado por error en una rama equivocada o para adoptar mejoras puntuales de otros colaboradores de forma quirúrgica, manteniendo así un control total sobre qué cambios exactos entran en tu historial.

16. Reparación y Reversión de cambios

- **git reset --soft HEAD~1** : Deshace el último guardado (commit) pero mantiene todos tus cambios listos en el Staging Area.
Se usa cuando acabas de hacer un commit y te das cuenta de que quieres agregar un archivo más o corregir el mensaje sin perder el trabajo que ya habías preparado.
- **git reset --mixed HEAD~1** : Deshace el último commit y saca los archivos del área de preparación, dejándolos como cambios sin guardar en tu carpeta.
Es el modo por defecto y se usa cuando quieres "desarmar" por completo tu último trabajo para reorganizar qué archivos vas a guardar y cuáles no.
- **git reset --hard HEAD~1** : Borra definitivamente el último commit y elimina todos los cambios realizados en los archivos, regresándolos al estado exacto del penúltimo guardado.
Se usa únicamente cuando cometiste un error catastrófico y quieres descartar absolutamente todo lo último que hiciste para empezar de nuevo desde un punto seguro.
- **git revert <HASH>** : Deshace los cambios introducidos por un commit específico mediante la creación automática de un nuevo commit inverso que neutraliza su contenido, a diferencia de otras herramientas que borran el pasado, este comando preserva la integridad del historial, lo que lo convierte en el método estándar para corregir errores en ramas compartidas o públicas donde no se debe alterar la línea de tiempo.
Se utiliza principalmente para anular fallos en producción de forma segura, permitiendo que el equipo vea documentado exactamente cuándo y por qué se dio marcha atrás a una funcionalidad sin causar conflictos de sincronización a los demás colaboradores.
- **git revert HEAD~2..HEAD** : Crea automáticamente los commits necesarios para deshacer los últimos dos cambios realizados en la rama actual.
Se usa cuando detectas que las últimas acciones fueron incorrectas y necesitas volver rápidamente a un estado anterior de forma limpia y segura para el resto del equipo.

17. Etiquetas

- **git tag** : Muestra una lista de todas las etiquetas o versiones que se han creado en el proyecto.
Se usa para consultar rápidamente los puntos importantes de la historia, como los lanzamientos oficiales (v1.0, v2.1, etc.), permitiéndote saber qué versiones han sido publicadas hasta la fecha.
- **git tag -a** : Crea una etiqueta "anotada", que incluye el nombre del creador, la fecha y un mensaje descriptivo, similar a un commit.
Se usa para marcar hitos fundamentales en el desarrollo, como el cierre de una versión de producción, proporcionando información extra sobre por qué se marcó ese punto específico.

- **git describe** : Busca el commit actual y devuelve una descripción legible basada en la etiqueta más cercana y la cantidad de cambios posteriores.
Se usa cuando estás trabajando en el código y necesitas saber rápidamente qué versión tienes como base o qué tan lejos estás del último lanzamiento oficial.
- **git push origin --tags** : Sube todas las etiquetas creadas localmente hacia el servidor remoto (como GitHub).
Se usa porque los comandos de "push" normales no envían las etiquetas automáticamente, es el paso final para compartir con el equipo o con el público las versiones terminadas del software.

18. Patrón de historial

- **Historia lineal vs ramificada** : Una historia lineal mantiene una secuencia continua de cambios de principio a fin sin cruces (lograda con Rebase), mientras que la ramificada muestra visualmente dónde se abrieron y unieron diferentes tareas (lograda con Merge).
Se usa la lineal para proyectos que buscan limpieza absoluta y la ramificada para proyectos que prefieren ver exactamente cuándo y cómo colaboró cada persona.
- **Rebase vs merge** : Merge une dos ramas creando un nuevo "commit de unión" que conserva todo el historial original, mientras que el Rebase mueve tus cambios al final de la otra rama para que parezca que siempre trabajaste sobre lo más nuevo.
Se usa Merge para integrar funciones terminadas de forma segura y Rebase para actualizar tu trabajo local con lo que otros subieron sin ensuciar el historial.
- **Revert vs reset** : Revert crea un nuevo commit que deshace cambios anteriores sin borrar el historial, mientras que el Reset borra directamente los commits del historial como si nunca hubieran existido.
Se usa Revert en repositorios compartidos (GitHub) para no romper el trabajo de otros, y Reset solo en tu trabajo local privado para corregir errores propios antes de compartirlos.
- **Detached HEAD** : Estado en el que el puntero HEAD apunta directamente a un commit específico en lugar de a una rama, se entra en este estado cuando usas "git checkout" para ver una versión antigua del código.
Se usa para explorar el pasado del proyecto o hacer pruebas rápidas sin afectar las ramas actuales, pero es importante saber que lo que guardes aquí se perderá si no se crea una rama nueva.
- **Conflictos y tracking** : Un conflicto ocurre cuando dos personas modifican la misma línea del mismo archivo y Git no sabe cuál elegir; el Tracking es la relación de "seguimiento" entre tu rama local y la del servidor.
Los conflictos se resuelven manualmente editando el archivo afectado, y el tracking se usa para que comandos como git pull o push sepan exactamente a dónde enviar la información sin preguntar.⁴
- **Cherry-pick selectivo** : Acción de extraer un cambio puntual de una rama y pegarlo en otra, ignorando el resto de los archivos de esa rama.

Se usa cuando necesitas una corrección urgente que está en una rama de prueba, pero no quieres llevarte todas las funciones experimentales que aún no están listas para producción.